

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D043.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 043 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

yfinance 拉数据

// yfinance

KEY CONCEPTS · 三大要点

- ▶ ticker
- ▶ history
- ▶ actions

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面几节用的都是国内的 baostock。这一节把视野放到全球:yfinance 是一个免费、不用注册、不用 token 的小工具,一行就能把美股(还有港股、外汇、加密、指数)的开高低收成交量拉回来。我们用它批量拉英伟达、特斯拉、微软、亚马逊、Meta 这五只人人都听过的美股,比一比5 年谁涨得最猛,顺便讲清两个新手必踩的坑:复权不做会被拆股骗出一个假暴跌、批量拉数据不限速会被网站封。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 会用 yf.Ticker(代码).history(period='5y') 一行拿到一只股票多年的开高低收成交量
- 02 会用循环 + time.sleep 限速批量拉一篮子股票,并用 dict 构造合并表防止列错位
- 03 理解复权(auto_adjust)的重要性:不复权会把拆股当成暴跌,算收益全错
- 04 会用 actions 查一只股票的分红和拆股历史
- 05 会用 isna 校验缺失值,知道停牌、上市晚会留下空洞,要按市场对齐后再算

02 · CONTEXT · 历史与背景

老张手动下了五个 CSV,还把英伟达的“暴跌”当成了崩盘

// 不读历史的策略走不远

老张想验证一个简单想法:这五只美国科技龙头,5 年前各买10000 块,今天各值多少?他老老实实跑去财经网站,一只一只点“导出 CSV”,下了五个文件,光是点鼠标就点到手酸。更糟的是,他下英伟达时随手选了“不复权”的版本,打开一看,2024 年某一天股价从 1000 多美元直接跌到 100 出头,他吓出一身冷汗,以为公司出了大事差点割肉——其实那天是英伟达一拆十,1 股变 10 股、股价自然除以 10,公司啥事没有。后来他学了 yfinance,一个循环就把五只股票5 年数据全拉回来,而且默认就是复权的连续价格,再没被假暴跌吓到。从手动点五个 CSV,到一段代码全自动,这就是数据工具帮你省下的时间和踩过的坑。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

yfinance:免费拉全球行情的小工具

yfinance 是一个开源 Python 包,背后连的是 Yahoo Finance 的公开数据,免费、不用注册、不用申请 key。一行 `yf.Ticker('NVDA').history(period='5y')` 就能拿到英伟达5 年的开高低收成交量。除了美股,港股(代码后缀 .HK)、A 股(.SS/.SZ)、外汇、加密货币、指数都能拉,是入门拿数据最省事的选择。

CONCEPT_02

history 与 download 两种拉法

拉一只股票用 `yf.Ticker('代码').history(period='5y')`,返回干净的单层列(Open/High/Low/Close/Volume)。一次拉一篮子可以用 `yf.download(['NVDA','TSLA'], period='5y')`,但它返回的是双层列(指标 × 代码),新手容易取错列。本节用循环 + dict 构造的方式批量拉,列名和数据一一对应,稳妥不出错。period 可以写 '1y'、'5y'、'max',也可以用 start/end 指定起止日期。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

复权 auto_adjust:不做会被拆股骗

公司会分红、会拆股,这些会让原始股价跳一下,但你的真实收益并没变,复权就是把这些影响抹平。yfinance 这里有个细节要分清:它返回的 Close 默认已经做了拆股调整(所以你看不到拆股断崖,Yahoo 替你处理了),而 Adj Close 是在此基础上把分红也算进去的「真复权价」。设 auto_adjust=True 时,拿到的 Close 就等于这个真复权价。算收益、做回测一律用复权价。提醒:有些数据源(比如下一节 akshare 的不复权选项)给的是完全没调过的原始价,那才会看到拆股当天的断崖,所以拉数据务必显式选复权。

CONCEPT_04

actions:分红和拆股记录

yf.Ticker('NVDA').actions 返回这只股票历史上的分红(Dividends)和拆股(Stock Splits)记录。拆股那一列的 10.0 就表示一拆十。知道这些事件发生在哪天,能帮你理解原始价格为什么会跳、复权为什么必要。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

批量限速 + 缺失校验

批量拉几十只股票时,别一口气连续请求,否则容易被网站当成爬虫限流甚至临时封。规矩做法是每拉一只 `time.sleep(1)` 歇一下。拉回来还要用 `isna().sum()` 检查缺失:有的股票上市晚、有的停过牌,会留下空洞。把不同股票拼成一张表后,常用 `dropna` 只保留大家都有数据的交易日再算(注意跨市场要分市场对齐,本节全是美股可统一处理)。

yfinance — Ticker.history 一行拉行情 / 批量循环 + 限速 / auto_adjust 复权 / actions 分红拆股 / 缺失校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_043_yfinance.py — yfinance:一行拉美股 / Ticker.history 与 download / 复权
auto_adjust / actions 分红拆股 / 批量循环 + 限速 / 缺失校验
# 用 yfinance 免费拉一篮子美股(英伟达/特斯拉/微软/亚马逊/Meta)五年行情,对比涨幅、看复权的重要性
# 数据:yfinance(免费拉全球股票,无需 token)
# 依赖:yfinance(已装)。注意:国内访问 yahoo 偶尔不稳,失败多重试或挂代理
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

pd.set_option('display.width', 140)
plt.rcParams['axes.unicode_minus'] = False
# 中文名 -> 美股代码(用 dict 显式映射,后面构造表时绝不会列错位 — 铁律 39)
TICKERS = {'英伟达': 'NVDA', '特斯拉': 'TSLA', '微软': 'MSFT', '亚马逊': 'AMZN',
'Meta': 'META'}

# =====
print('==== 1. 单只股票:Ticker.history 一行拿到开高低收成交量 ====')
nvda = yf.Ticker('NVDA')
hist = nvda.history(period='5y', auto_adjust=True) # period='5y' 直接拿五年
print(f'英伟达 五年日线 {len(hist)} 条')
print(hist[['Open', 'High', 'Low', 'Close', 'Volume']].tail(3).round(2))

print('\n==== 2. actions:分红和拆股记录 ====')
print('英伟达近几次分红/拆股(actions):')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/4)

yfinance — Ticker.history 一行拉行情 / 批量循环 + 限速 / auto_adjust 复权 / actions 分红拆股 / 缺失校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(nvda.actions.tail(4))
print('Stock Splits 那一列里的 10.0 表示一拆十:1 股变 10 股,股价相应除以 10')

# =====
print('\n==== 3. 批量拉一篮子:循环 + 限速(别被 yahoo 封)====')
closes = {}
for name, tk in TICKERS.items():
    h = yf.Ticker(tk).history(period='5y', auto_adjust=True)
    closes[name] = h['Close']
print(f'{name}({tk}) 拉到 {len(h)} 天')
time.sleep(1) # 每拉一只歇 1 秒,批量拉几十只时尤其重要,否则容易被限流
# 用 dict 构造,列名就是中文名,和数据一一对应,绝不会像 columns= 赋值那样错位(铁律 39)
close = pd.DataFrame(closes)
close.index = close.index.tz_localize(None) # 去掉时区,方便后面对齐和画图
print(f'\n合并后行情表 {close.shape[0]} 行 × {close.shape[1]} 列')

print('\n==== 4. 缺失值校验:停牌、上市晚都会留下空洞 ====')
print('各只股票缺失天数(NaN):')
print(close.isna().sum())
close = close.dropna() # 按市场统一对齐:都有数据的交易日才留(本篮子全是美股,可统一 dropna)
print(f'对齐后剩 {len(close)} 个共同交易日 {close.index[0].date()} → {close.index[-1].date()}')

# =====
print('\n==== 5. 五年涨幅大比拼(净值=各自起点归 1)====')
nv = close / close.iloc[0]
final = nv.iloc[-1].sort_values(ascending=False)
print('五年净值(起点 1,数字=翻了几倍):')
for name, v in final.items():
    print(f' {name:6} {v:6.2f} 倍')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/4)

yfinance — Ticker.history 一行拉行情 / 批量循环 + 限速 / auto_adjust 复权 / actions 分红拆股 / 缺失校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
fig, ax = plt.subplots(figsize=(13, 6))
for name in final.index:
    ax.plot(nv.index, nv[name], lw=1.4, label=f'{name} {final[name]:.1f}x')
    ax.set_title('五只美股五年净值对比(各自起点归 1)'); ax.set_ylabel('净值(倍)')
    ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.savefig('chart_01.png', dpi=120); plt.close()
print('图1(五年净值对比)已保存')

# =====
print('\n==== 6. 复权到底复了什么:Close 已调拆股,Adj Close 再把分红也算进去 ====')
m = yf.Ticker('MSFT').history(period='5y', auto_adjust=False)
m.index = m.index.tz_localize(None)
close_split = m['Close'] # Yahoo 默认已做拆股调整,但没把分红算进去
adj = m['Adj Close'] # 拆股 + 分红都调好的真复权价(= auto_adjust=True 时的 Close)
gap0 = (close_split.iloc[0] / adj.iloc[0] - 1) * 100
fig, ax = plt.subplots(figsize=(13, 5.5))
ax.plot(close_split.index, close_split, color='#bf616a', lw=1.3, label='Close(只调拆股,没算分红)')
ax.plot(adj.index, adj, color='#5e81ac', lw=1.3, label='Adj Close(拆股 + 分红,真复权价)')
ax.set_title(f'微软:Close vs Adj Close(起点差 {gap0:.1f}% 就是 5 年累计分红的贡献)')
ax.set_ylabel('价格(美元)'); ax.legend()
plt.tight_layout(); plt.savefig('chart_02.png', dpi=120); plt.close()
print(f'图2(复权对比)已保存:起点 Adj Close 比 Close 低 {gap0:.1f}%,这就是分红被算进真实收益')
print('拆股(如英伟达 2024 一拆十)Yahoo 的 Close 已自动调好,所以这里看不到断崖;')
print("要你留心的是分红 — Adj Close 才是含分红的真复权价,auto_adjust=True 拿到的 Close 就等于它")
print("下一节 akshare 拉 A 股时 adjust='' 给的是没调过的原始价,那才会看到拆股断崖,务必显式选复权")

# =====
```

```
print('\n==== 7. 单只 K 线 + 成交量(英伟达近一年)====')  
one = yf.Ticker('NVDA').history(period='1y', auto_adjust=True)  
one.index = one.index.tz_localize(None)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/4)

yfinance — Ticker.history 一行拉行情 / 批量循环 + 限速 / auto_adjust 复权 / actions 分红拆股 / 缺失校验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

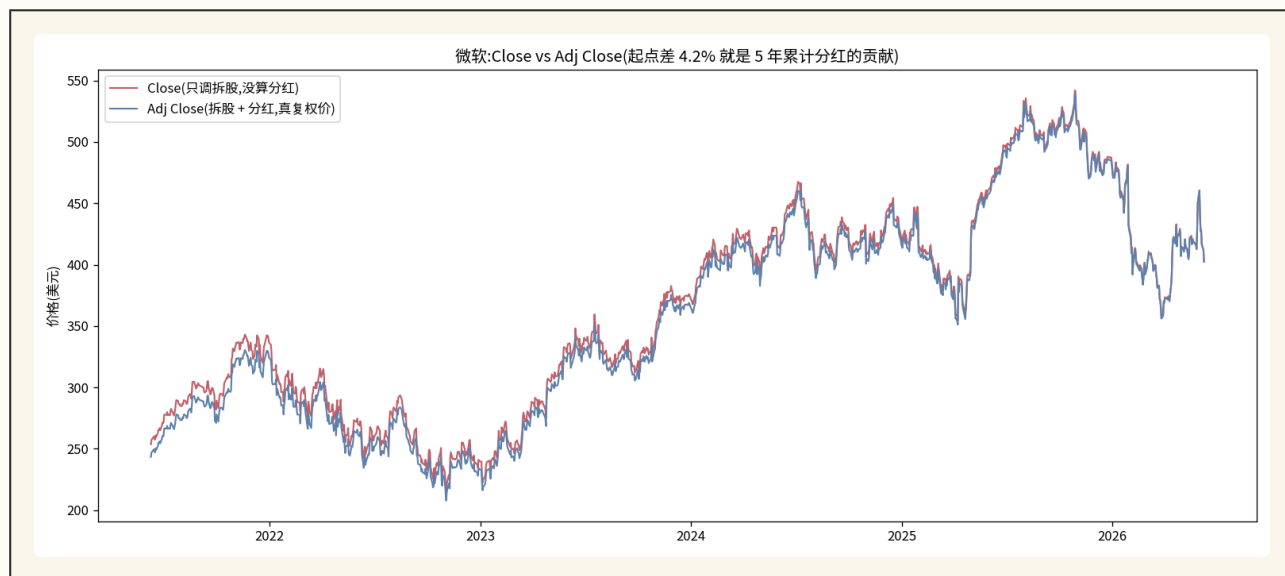
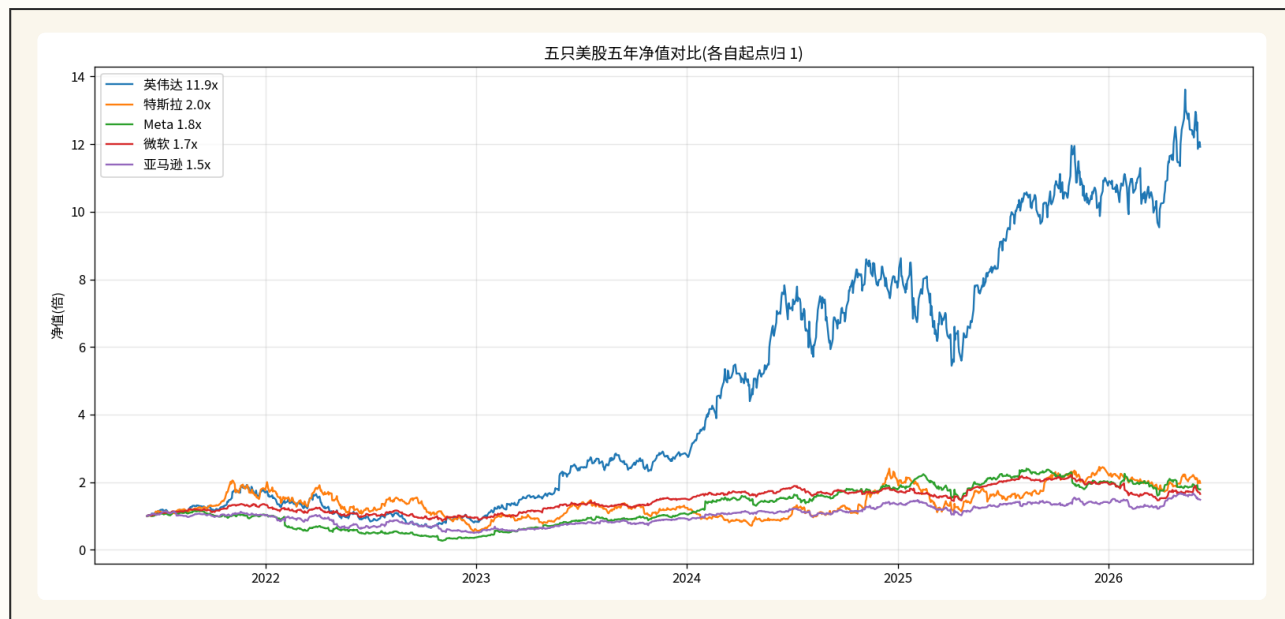
```
fig, (axp, axv) = plt.subplots(2, 1, figsize=(13, 7), sharex=True,
                               gridspec_kw={'height_ratios': [3, 1]})
axp.plot(one.index, one['Close'], color='#4c566a', lw=1.2)
axp.plot(one.index, one['Close'].rolling(20).mean(), color='#d08770', lw=1.1,
         label='20 日均线')
axp.set_title('英伟达 近一年 收盘 + 20 日均线 / 成交量'); axp.set_ylabel('价格(美元)');
axp.legend()
axv.bar(one.index, one['Volume'] / 1e6, color='#b48ead', width=1.0)
axv.set_ylabel('成交量(百万股)')
plt.tight_layout(); plt.savefig('chart_03.png', dpi=120); plt.close()
print('图3(单只 K 线 + 成交量)已保存')

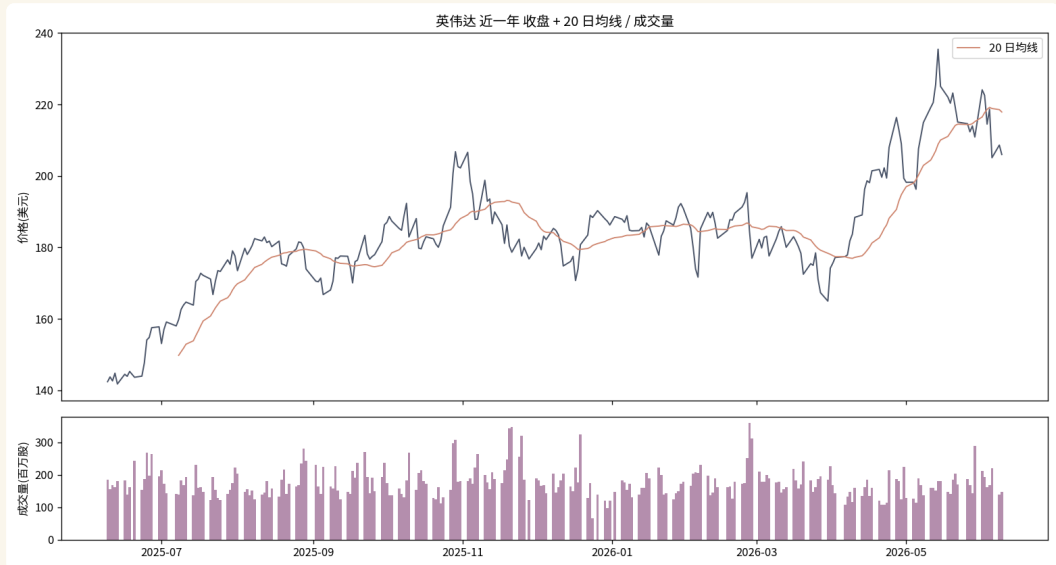
print(f'\n[小结] 五年最强 {final.index[0]} {final.iloc[0]:.1f} 倍,最弱
{final.index[-1]} {final.iloc[-1]:.1f} 倍')
print('\n[done] 3 张图已生成,output.txt 见上方全部打印')
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





绘图数据(5 只美股 5 年)

英伟达/特斯拉/微软/亚马逊/
Meta 各 1256 个交易日;区
间 2021-06-09 →
2026-06-09;一行
`history(period='5y')`
批量拉回

五年净值排名(起点归 1)

英伟达 11.9 倍 · 特斯拉
2.0 倍 · Meta 1.8 倍 ·
微软 1.7 倍 · 亚马逊 1.5
倍;同板块冠亚军差近 8 倍

复权命门(微软 Close vs
Adj Close)

Close 默认已调拆股(看不到
断崖),Adj Close 再把分红
算进去=真复权价;两条线终点
重合、起点差 4.2%,这
4.2% 就是 5 年累计分红

`actions` 分红拆股

`yf.Ticker('NVDA').acti
ons` 列出历史分红(每次约
0.01~0.25 美元)和拆股
(2024 一拆十,Stock
Splits 那列的 10.0);知道
事件在哪天才懂原始价为何跳

批量限速 + 防列错位

循环每只 `time.sleep(1)`
防被封;合并用 dict 构造
(`{name: raw[ticker]}`)
对齐列名,绝不靠列顺序(铁律
39);跨市场拼接先
`tz_localize(None)` 去时
区

英伟达近一年走势(第三张图)

收盘价折线 + 20 日均线 +
成交量副图;价格从约 143
美元一路震荡上行到 220 美
元一带,数据格式与前两节画
图完全通用

► 01 免费、免注册,一行拉回全球行情

yfinance 背后连的是雅虎财经的公开数据,不用注册、不用申请任何凭证、一分钱不花,一行 `history(period='5y')` 就拿到英伟达 5 年共 1256 个交易日的开高低收成交量。更关键的是它不挑市场,美股、港股(代码加 .HK)、外汇、加密货币(如 BTC-USD)、各种指数,同一套写法都能拉。十几年前,普通散户想拿到全球市场的连续行情几乎不可能,如今一个免费小工具就替你把这扇门推开了——这是数据民主化实实在在的红利。

► 02 ★ 复权才是本课命门:拆股骗你假暴跌,分红让你少算钱

这是新手最致命也最常踩的坑。一家公司分红、拆股都会让股价跳一下,但你的真实资产没变,复权就是把这些跳变抹平、还原成能真实反映赚赔的连续价格。要分清两层:yfinance 给的 Close 默认已经把拆股调好了,所以你在它这儿看不到英伟达一拆十那种从 1000 跌到 100 的吓人断崖——真正要你留心的是分红。Adj Close 才是把分红也算进去的真复权价。微软那张图两条线终点重合、起点差 4.2%,这 4.2% 就是 5 年累计分红。结论八个字:算收益、做回测,一律用复权价。

► 03 换数据源第一件事:搞清给的是复权价还是原始价

不同数据源对复权的处理口径不一样,这点不弄清就会栽跟头。yfinance 贴心地默认替你调好了拆股;但下一节拉 A 股要用的 akshare,它的 `adjust=""` 选项给的是完全没调过的原始价,那才会实实在在看到拆股当天的断崖。所以每换一个数据源,先问一句:它给我的到底是复权价还是原始价?老张当年就是在财经网站随手下了不复权版本,被英伟达一拆十的假暴跌吓得差点割肉。

► 04 5 年净值差近 8 倍:选对赛道不够,选对个股才拉开差距

把 5 只票起点都归 1 再看 5 年后:英伟达 11.9 倍一骑绝尘冲上天,而特斯拉 2.0、Meta 1.8、微软 1.7、亚马逊只有 1.5 倍。哪怕你 5 年前有先见之明、只押美国科技龙头,买中英伟达和买成亚马逊,结果也差了 78 倍。这说明在同一个看起来都很牛的板块里,选股的差异大到惊人。别以为买进好赛道就高枕无忧——具体买哪一只,才是真正拉开差距的地方。而且事后看冠军容易,5 年前你敢不敢用大仓位押、又拿不拿得住,才是真问题。

► 05 批量拉数据的工程纪律:限速、防错位、去时区、能回退

拉一篮子股票有 4 条纪律。一,循环里每只之间 `time.sleep(1)` 歇一下,否则被雅虎当爬虫限流甚至封 IP——这既是礼貌也是稳定拿数据的前提。二,合并成大表时用 dict 构造(`{name: raw[ticker]}`)让列名和股票牢牢绑定,绝不靠列顺序硬安名字,否则英伟达的数据可能被贴上特斯拉的标签(铁律 39 老坑)。三,yfinance 索引带美东时区,跟国内数据拼接前先 `tz_localize(None)` 去掉时区。四,国内访问雅虎不稳,要 `try/except` 失败就回退预存的本地 CSV(铁律 55),别让一次掉线卡死整个程序。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

美股

NVDA/TSLA/MSFT/
AMZN/META

批量拉五只美股科技龙头5 年行情,净值归一比谁涨得最猛

复权

MSFT

微软 Close vs Adj Close:起点差约 4% 就是 5 年累计分红的贡献,Adj Close 才是含分红的真复权价

公司事件

NVDA

用 actions 查分红和拆股历史,理解原始价格为何跳变

跨市场拓展

0700.HK / BTC-
USD

同一套写法把代码换成港股腾讯 0700.HK 或加密 BTC-USD 就能拉,接口通用

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

不复权算收益,全错

把 `auto_adjust` 设成 `False` 又直接拿 `Close` 去算长期收益,会漏掉分红(`Close` 没把分红算进去);在别的数据源用完全没调过的原始价,还会被拆股当天的跳变骗成假暴跌。算收益、做回测一律用复权价(`auto_adjust=True`,或取 `Adj Close`),这是最致命也最常见的错。

△ 02

批量拉不限速被封

写个循环连续猛拉几十上百只,Yahoo 会把你当爬虫限流,轻则拿到空数据、重则临时封 IP。每只之间 `time.sleep` 歇一下,失败了重试,是基本礼貌也是稳定拉数据的前提。

△ 03

download 多股票列取错

`yf.download` 多只股票返回双层列(指标 × 代码),直接 `df['Close']` 拿到的是所有股票的收盘,再用 `columns=` 强行改名极易错位(铁律 39 老坑)。要么像本节用 `dict` 构造,要么 `raw['Close']`[代码] 按名字取,绝不靠列顺序。

△ 04

时区不统一对不齐

`yfinance` 返回的索引带美东时区,和国内数据或别的来源拼接时会因时区对不齐。统一用 `index.tz_localize(None)` 去掉时区再对齐、再画图,省得踩坑。

△ 05

国内网络不稳直接卡死

国内访问 Yahoo 偶尔超时或被墙,代码会卡住或报错。正经做法是包一层 `try/except`,失败就回退到预先下载好的本地 CSV(本节中国版就是这么写的),别让联网失败把整个 notebook 卡死。

yfinance 拉数据实战 6 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

01 算收益一律用复权价 — 保持 `auto_adjust=True`,别用不复权 `Close`。

02 批量拉必限速 — 每只 `time.sleep` 一下,失败重试,别被当爬虫。

03 合并多股票用 `dict` 构造 — 列名对应数据,绝不用 `columns=` 按顺序赋值(铁律 39)。

04 拉完先查缺失 — `isna().sum()` 看空洞,按市场对齐后再算。

05 时区先归一 — `tz_localize(None)` 去时区,再对齐、再画图。

06 国内联网包 `try/except` + `CSV` 回退 — 失败不卡死,读预下载的本地数据。

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 yfinance 免费、免注册, 一行 `history(period='5y')` 就拉到一只股票多年的开高低收成
交量。
- 02 批量拉一篮子用循环 + `time.sleep` 限速, 合并表用 `dict` 构造防止列错位(铁律 39)。
- 03 复权是命门: `Close` 默认已调拆股, `Adj Close` 再把分红算进去才是真复权价; 算收益、做
回测用复权价(`auto_adjust=True`)。
- 04 `actions` 查分红和拆股历史, 帮你理解原始价格为什么会跳。
- 05 `isna` 校验缺失: 上市晚、停牌会留空洞, 跨市场要分市场对齐再算。
- 06 国内网络不稳, 中国版用 `yfinance` 优先 + 本地 `CSV` 回退, 联网失败不卡死(铁律 55)。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用 yfinance 拉一只股票5 年的日线,最简单的一行怎么写?(提示:yf.Ticker('NVDA').history(period='5y'),返回干净的单层列 Open/High/Low/Close/Volume;period 也可写 '1y'、'max',或用 start/end 指定起止。)

Q02 yfinance 里 Close 和 Adj Close 有什么区别,算收益用哪个?(提示:Close 默认已做拆股调整但没算分红;Adj Close 在它基础上把分红也调进去,是含分红的真复权价。设 auto_adjust=True 时 Close 就等于 Adj Close。算收益、做回测一律用复权价,否则会漏掉分红、或在别的数据源被拆股跳变骗。)

Q03 批量拉一篮子股票要注意什么?怎么合并成一张表不出错?(提示:循环里每只 time.sleep 限速别被封;合并时用 dict 构造 DataFrame,列名就是股票名、和数据一一对应,绝不用 columns= 按顺序硬赋值,否则会列错位——铁律 39。)

Q04 国内访问 Yahoo 不稳定,代码该怎么写才不会卡死?(提示:把拉数据包进 try/except,先尝试 yfinance,失败就回退去读预先下载好的本地 CSV;这就是铁律 55 的双路径写法,联网失败也能跑完。)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- yfinance 官方文档 / GitHub README(ranaroussi, 2024)– Ticker、history、download、actions 的权威用法
- Yahoo Finance 《What is adjusted close?》(2024)– 复权价的定义与为什么算收益要用它
- Wes McKinney 《Python for Data Analysis》(2022, O'Reilly)– 多张行情表的对齐、合并与缺失处理
- Yves Hilpisch 《Python for Finance》(2018, O'Reilly)– 拿金融数据做收益与净值分析的完整流程
- Marcos López de Prado 《Advances in Financial Machine Learning》(2018, Wiley)– 第一章谈数据质量,复权与对齐为何是回测的地基

NEXT LECTURE · 下一节预告

DAY 044 akshare 拉 A 股

yfinance 拉海外方便,但 A 股的行情和财务它就不灵了。下一节请出国内数据的瑞士军刀 akshare: 免费拉沪深 300 成分股、个股财报、宏观甚至另类数据,看看做 A 股研究怎么零成本把数据备齐。